Customer Review Auto-Sorter

A Python NLP automation project: sentiment analysis, topic categorization and real-time alerts on 100,000 customer reviews

1. Project Overview

A shop receives thousands of customer reviews and nobody can read them all. This project builds a small program that reads them automatically, decides whether each review is happy, neutral or angry, identifies what it is about (delivery, price, quality or service), and raises an alarm the moment an angry review arrives. Angry reviews are saved to a file that acts as a ready-made to-do list for the support team.

Step	What the program does	Concept
1	Creates 100,000 sample reviews	Large dataset generation
2	Reads the file in small pieces (chunks)	Memory-efficient data processing
3	Classifies each review as happy, neutral or angry	Sentiment analysis (NLP)
4	Detects the topic of each review	Keyword-based categorization (NLP)
5	Watches new reviews and alerts on angry ones	Automation and real-time monitoring

Tech stack: Python 3, Pandas, Matplotlib, VADER Sentiment. **Install:**

```
pip install pandas matplotlib vaderSentiment
```

Pipeline

```
reviews.csv --> read in chunks --> mood --> topic --> chart
                                                    \
new live reviews ---------------------> mood + topic --> ALARM + urgent_reviews.csv
```

2. LinkedIn Post (ready to copy)

Built a Customer Review Auto-Sorter in Python: turning 100,000 reviews into instant, actionable insights

Businesses receive thousands of customer reviews, but nobody has time to read them all. So I built a small NLP automation project that reads them for us, sorts them, and raises an alarm when a customer is unhappy.

What the project does

- Generates a large dataset of 100,000 sample reviews
- Processes data in chunks of 20,000 rows, so even huge files never overload memory
- Runs sentiment analysis with VADER to classify each review as happy, neutral or angry
- Categorizes reviews by topic (delivery, price, quality, service) using keyword-based NLP
- Visualizes results with a topic-vs-mood chart showing exactly where customers are unhappy
- Simulates a real-time review feed that triggers alerts for angry reviews and saves them to a CSV, a ready-made to-do list for the support team

Key takeaway
No human reads a single review, yet the right people see the angriest ones immediately. That is the power of automation combined with NLP.

Tech stack
Python | Pandas | Matplotlib | VADER Sentiment | NLP | Automation

Built to scale
- Swap the sample data for real customer reviews (any CSV with a "review" column)
- Connect a live API or database feed
- Send alerts to Slack or email via webhooks
- Upgrade to transformer models for sarcasm and multilingual text

This project taught me how chunked data processing, sentiment analysis and real-time monitoring come together in a practical business workflow. I'd love your feedback: how would you extend this?

GitHub: [add your link here]

#Python #NLP #DataScience #Automation #SentimentAnalysis #MachineLearning #DataAnalytics #Pandas #PortfolioProject #LearningInPublic

Posting tips: attach the topic-vs-mood bar chart as an image, and place your GitHub link in the first comment for better reach.

3. Code Walkthrough with Descriptions

Step 0: Load the tools

Imports the libraries and creates the sentiment analyzer used throughout the project.

```
import random, time
import pandas as pd
import matplotlib.pyplot as plt
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

reader = SentimentIntensityAnalyzer()
random.seed(1)
```

- **pandas** handles tables, **matplotlib** draws charts, **random** and **time** support data generation and the live-feed delay.
- **VADER** is a free, rule-based tool that scores text from -1 (very negative) to +1 (very positive). No model training is needed.
- **random.seed(1)** makes the random data identical on every run, so results are reproducible.

Step 1: Create 100,000 sample reviews

Builds a large dataset by combining random topic sentences with random good or bad words.

```
topics = {
    "delivery": ["The delivery was {}", "Shipping was {}"],
    "price":    ["The price is {}", "The cost is {}"],
    "quality":  ["The product quality is {}", "Build quality feels {}"],
    "service":  ["Customer service was {}", "The service team was {}"],
}
good = ["excellent", "great", "fantastic", "wonderful", "amazing"]
bad  = ["terrible", "awful", "horrible", "disappointing", "very poor"]

def make_review():
    topic = random.choice(list(topics))
    mood = random.choice([good, bad])
    return random.choice(topics[topic]).format(random.choice(mood))

pd.DataFrame({"review": [make_review() for _ in range(100_000)]}
            ).to_csv("reviews.csv", index=False)
```

- **topics** maps each topic to sentence templates with a {} placeholder.
- **make_review()** picks a random topic, a random good or bad word set, and fills the template (for example, "Shipping was awful").
- A list comprehension creates 100,000 reviews, and **to_csv** saves them to reviews.csv.
- To use your own data, skip this step. Your file only needs a column named **review**.

Steps 2 and 3: Read in chunks and score the mood

Reads the file in small pieces and labels each review with a mood.

```
def mood(text):
    score = reader.polarity_scores(text)["compound"]
    if score >= 0.3:
        return "happy"
    if score <= -0.3:
        return "angry"
    return "neutral"

parts = []
for piece in pd.read_csv("reviews.csv", chunksize=20_000):
    piece["mood"] = piece["review"].map(mood)
    parts.append(piece)
    print(f"Processed {len(piece):,} reviews")

data = pd.concat(parts)
```

- **polarity_scores(text)["compound"]** returns one overall score between -1 and +1.
- **Thresholds:** a score of 0.3 or above is happy, -0.3 or below is angry, and anything between is neutral.
- **chunksize=20_000** makes pandas load 20,000 rows at a time, so a huge file never has to fit in memory at once.
- **.map(mood)** applies the helper to every review and stores the result in a new **mood** column.
- **pd.concat(parts)** joins the processed chunks into one table.

Step 4: Find out what each review is about

Assigns a topic using keyword matching, then counts and charts topics by mood.

```
keywords = {
    "delivery": ["delivery", "shipping"],
    "price":    ["price", "cost"],
    "quality":  ["quality", "build"],
    "service":  ["service"],
}

def topic_of(text):
    text = text.lower()
    for topic, words in keywords.items():
        if any(w in text for w in words):
            return topic
    return "other"

data["topic"] = data["review"].map(topic_of)

table = pd.crosstab(data["topic"], data["mood"])
print(table)
table.plot.bar(title="Reviews by topic and mood",
               color=["#d9534f", "#5cb85c"])
plt.ylabel("Number of reviews"); plt.xticks(rotation=0); plt.show()
```

- **keywords** lists trigger words for each topic. New topics can be added by adding a line.
- **topic_of()** lowercases the text, checks each topic's words, returns the first match, or "other" if none match.
- **pd.crosstab** counts reviews for every topic and mood combination.
- The bar chart shows where customers are unhappy, which is the main insight of the project.

Result: topic vs mood counts

Topic	Angry	Happy
delivery	12,434	12,371
price	12,355	12,647
quality	12,515	12,609
service	12,522	12,547

The counts are nearly balanced because the sample data is randomly generated. With real reviews, the imbalance reveals the problem areas.

Step 5: Real-time automation and alarms

Simulates a live review feed, checks each new review and saves the angry ones.

```
def new_reviews(n=5):
    # Pretend live feed. Replace with an API, database or file check.
    return [make_review() for _ in range(n)]

urgent = []
for round_number in range(1, 6):          # use "while True:" to run forever
    print(f"--- Check {round_number} ---")
    for text in new_reviews():
        feeling = mood(text)
        if feeling == "angry":
            urgent.append({"review": text, "topic": topic_of(text)})
            print(f"  ALARM [{topic_of(text)}]: {text}")
    time.sleep(1)                         # wait 1 second between checks

pd.DataFrame(urgent).to_csv("urgent_reviews.csv", index=False)
print(f"Saved {len(urgent)} urgent reviews to urgent_reviews.csv")
```

- **new_reviews()** simulates five new reviews arriving per check and can later be replaced by a real data source.
- The outer **for** loop is the watcher: check, handle, wait one second, repeat.
- Every new review reuses the same **mood()** and **topic_of()** helpers.
- Angry reviews trigger an **ALARM** message and are added to the urgent list.
- At the end, **urgent_reviews.csv** is saved as an action list for the support team. In the sample run, 13 urgent reviews were found.

4. Make It Your Own

Goal	What to change
Use real reviews	Point Step 2 to your own CSV (needs a **review** column)
Use a real live feed	Rewrite **new_reviews()** to call an API, for example **requests.get(...)**
Get alerts in Slack or email	Replace the **print("ALARM...")** line with a webhook or email call
Run forever	Change **for round_number in range(1, 6):** to **while True:**
Add more topics	Add lines to the **keywords** dictionary
Handle sarcasm or other languages	Replace VADER with a transformers sentiment model